Midterm Review

- **Greedy Algorithms**: it solves problems when you want a maximal/minimal version of something. It doesn't think of the future, it wants the best choice now!
    - **Stable Matchmaking:** a set of doctors and hospitals each rank the other, based on preference. The hospital gets 'engaged' to the first doctor that approaches them. If another 'higher ranking' doctor comes along, then the first match is broken, and there is a new pair. This algorithm prefers whoever approaches first, as their preferences get prioritized
    - **Interval Scheduling**: as a student, you want to graduate as soon as possible. You want to take as many classes as possible; how do you choose? You cannot take overlapping classes
        - **Given a set of time intervals, find the largest non-overlapping set:**
            - Option 1: Brute Force: Look at all possible combinations of intervals to see which is the largest set of non-overlapping intervals O(n^2)
            - Option 2: Greedily pick the first class: remove all overlapping classes, then pick the available class
                - Problem: what if the earliest class is also the longest class? You lose out on picking more classes because you are stuck with a long class
            - Option 3: Greedily pick classes by shortest duration: shortest class may let you pick other classes quickly
                - Problem: What if the shortest class conflicts with many classes, thus not allowing you to pick those other classes
            - Option 4: Greedily choose classes with fewest conflicts: sort the classes based on conflicts and choose one with least conflicts.
                - Problem: class with least fewest conflicts could interfere with better combination
            - Option 5: Greedily choose classes that end the earliest: pick the class with the earliest end time, and remove all classes that conflict
                - Solution: it leaves you with the most amount of time in the rest of your day to pick other classes
    - **Interval Covering:** find a subcollection containing as few intervals as possible, such that the union of the intervals is covered.
        - **Given a set of intervals, find the smallest set of intervals that still cover the union:**
            - Option 1: Select Earliest Start time: Start J at 0, until all intervals cover [0, M] -> find the minimum c in [0, M] such that [c, M].

    - You look at the ‘uncovered’ section of intervals, and choose the earliest start of any intervals that cover that
    - Incorrect solution
    - M = 5, I = [[0, 2], [2, 5], [1, 3], [3, 5]]
    - Optimal solution here is 2 intervals [[0, 2], [2, 5]]
    - The algorithm takes 3.
        - J is empty, so c = 0. The only interval containing 0 is [0,2], which it must pick
        - now [0,2] is covered, so the next uncovered point is starting at 2. Intervals that contain that
        - points are [1, 3], [2, 5], [3, 5]. Earliest start is [1, 3].
        - Now [0,3] is covered, remaining intervals are [2,5] and [3,5]. the earliest start is [2,5]
- Option 2: Select Interval Maximizing Overlap Coverage: among all intervals in I that are not already contained in the union of the intervals in J, select one that covers as much as possible, i.e.. That maximizes (b – a) and add it to J
    - J = [], and you choose the longest interval that is not already covered by union of J. Then you pick from another set of intervals that cover the remainder of the union, picking the longest interval from that. Continue until you reach full coverage
    - Incorrect solution
    - M = 6, I = [[1, 5], [0, 3], [3, 6], [0, 2], [4, 6]]
    - Optimal solution here is 2 intervals [[0, 3], [3, 6]]
    - J starts empty, and picking the longest interval is [1,5].
    - Next, picking the next longest interval not already in coverage gives us either [0,3] or [3,6].
    - meaning there is still some coverage left, forcing us to pick another interval.
    - Which causes us to have 3 intervals.
- Option 3: Delete Earliest Redundant Interval: While there exists a redundant interval in J, one that is already contained in the union of the other intervals, remove the interval that finishes first
    - For each round, gather all intervals that are deemed ‘unnecessary’, as removing it would not affect the union coverage. From this list, remove the interval that finishes first.
    - Correct Solution

- Option 4: Delete Latest Redundant Interval: While there exists a redundant interval in J, one that is already contained in the union of the other intervals, remove the interval that finishes last
    - For each round, gather all intervals that are deemed 'unnecessary', as removing it would not affect the union coverage. From this list, remove the interval that finishes first.
    - Incorrect solution
    - Answer: M = 3 I = [[0, 1], [1, 2], [2, 3], [0, 3]]
    - Optimal solution here is 1 interval [[0, 3]]
    - We start with J = I
    - Every interval is redundant, so picking the latest finish time leads to deleting [0,3]
    - Now left with the remaining intervals, none of which are redundant. Meaning you don't delete anymore, leaving you with 3 intervals

- **Huffman Coding**: assigning a variable length codeword with the goal of minimizing the length of the text. The issue with codewords is you do not know when a codeword ends, hence we create prefix codes. Prefix codes is a code where no code word is a prefix of another. There's no confusion as to what each letter means
    - **Huffman Tree**: calculate the frequencies of each character, then sort them by highest to lowest. Choose two of the lowest character frequencies, combine them. The new combined frequency gets added to the list, while also removing the two we used. Now, we look at the new updated frequency list and again combine the two lowest frequencies.
        - The tree is created bottom up. Every time you take two frequencies, it gets added to the bottom of the tree as children
        - Facts about the best tree:
            - Every non-leaf node has 2 children
            - If frequencies of A is greater than frequencies of B, then
            - The two least frequent characters can be siblings
- **JPEG Compression:** Joint Photographic Experts Group and is a lossy compression algorithm that results in significantly smaller file sizes with little to no perceptible impact on picture quality and resolution.
    - **Run Length Encoding:** Given a set of integers, determine how many zeros occur before the next real number. Output is the integer, along with how many zeros were before it.
- **Most Frequent Difference:** Given a sequence of integers, we want to find the number of times the most frequent pairwise difference appears. Given the array

and a target integer δmode, count how many ordered pairs (i, j) with i ≠ j satisfy ai − aj = δmode.
    - **O(n^2) solution:** similar to the 2sum problem, where you can set up a double for-loop, ensure i is not equal to j. Check the difference between values[i] – values[j], if δmode == difference, increase the counter
    - **O(nlogn) solution:** sort the list, iterate through the list, calculate the target (values[i] – δmode). Bisect returns the first occurrence of target in list. So you find the range of first occurrence vs last occurrence, and since list is sorted, all numbers in between are frequencies of δmode.
    - **O(n) solution:** Create a hashmap of values and their frequency. Then iterate through list, get the target (values[i] – δmode), find the target in hashmap and increase counter. Remember to check if same index
  - **Cookie Distribution:** There are apartments in a building, and a staircase to go from one floor to the next. You start at the center and need to visit each friend in the fewest steps possible.
    - You calculate the center and then finalize the starting point. Also keep track of visited apartments
    - Iterate through all apartments, calculate distance from starting apt to every apt. Add this distance in a priority heap, with distance as key. Also keep a hashmap of distance for each apt
    - While the visited apts is one less than number of apartments, pop from heap, giving you the next shortest path.
      - Go through all apts, and calculate new distances from that apt to other apts, and if new distance is shorter, then update the hashmap
- **<u>Divide and Conquer:</u>** Divide: break each problem into subproblems; Conquer: each subproblem is solved recursively; recombine results
  - **MergeSort**: recursively split the list into two halves, then merge the results by combining the elements and choosing L1 vs L2 O(nlogn)
  - **Closest Pair of Points**: Given n points in a 2D plane, goal is to find the two points that are closest together
    - **O(n^2) solution:** compare all points with each other, keep track of shortest distance
    - **O(nlogn) solution:** Sort points by x-axis, find the median, recurse on both sides. Then compare across the strip, sort by y-axis, compare the shortest distance overall
      - First, you need to sort by x-axis. Then you call keep dividing the dataset into halves until you reach base case of length 3.
        - Find shortest distance between 2 or 3 points
      - Find the middle point, then separate the points in half. Keep splitting using recursion until you hit the base cases

        - This returns you the shortest delta distance between any two points on either side. But we still need to compare across the first split
    - We build the strip, which is current shortest distance (delta) away from the middle. If two points are beyond that, then it cannot be shorter than delta.
    - Once the strip is built with points (sorted by y-axis), divide the strip into delta/2 x delta/2 boxes.
        - At most one point in each box
        - Any point q that could be within delta of p is at most 2 rows above p. anything 3 or more rows above is more than delta away vertically
        - Why 11:
            - 3 rows (p's row and the 2 above it) × 4 squares per row = 12 squares
            - Leave out p's own square: 11 squares
            - Each square has at most 1 point, so at most 11 points
- **Master Theorem**: provides a fast, direct way to find the time complexity of Divide and Conquer relations.
    - **T(n) = a T (n / b) + f (n)**
        - n: size of input
        - a: number of subproblems in the recursion (a >= 1)
        - n / b : size of each subproblem (b > 1)
        - f(n): cost of the work done outside the recursive calls (splitting the problem and merging solution)
    - Leaf-dominated (work at the bottom is heavier)
        -

- **<u>Dynamic Programming:</u>** start by thinking about one item, either the optimal solution includes the item, or doesn't. If it does, then the remaining items are optimal on the subproblem w/ capacity minus the new item. Else if optimal solution does not contain the item, then the remaining items are optimal without this item.
    - **0/1 Knapsack Problem:** Input is n items, each with a value and weight, and capacity. Output is the maximal total value without exceeding capacity, the subset S that achieves highest value. Each item can only be included once
    - **Maximum sum of non-adjacent elements:** compute the maximum sum of non-adjacent elements.
        - Set up a recursion. If you pick the current element, then you cannot pick its neighbor. Else if you do not pick current element, then neighbor is

available. The base case is if you end up with one element, then you of course pick it.
        - You return the max recursion of picking vs not picking the current element. You pass the list with or without the current element to the recursion. You add to total of the 'pick' condition
        - To optimize, you can memoize the solution. You can add the current element to map, along with its calculated max of picking vs not picking current element. So if you ever come across the same number again, you can just use previous calculation

- **<u>Randomized Algorithms:</u>**
    - A random variable assigns a numerical value to each outcome of an experiment
        - For two dice, examples include:
            - The sum of the dice
            - The value shown on the first die
            - An indicator for doubles: 1 if doubles occur, and 0 otherwise.
        - In the hashing experiment, examples include
            - The number of colliding pairs
            - The number of keys in a particular bin
            - An indicator for whether there are no collisions.
        - Expected value
            - The expected value is the long-run average value of a random variable
    - **Hash Tables**: A hash table is a dictionary or map used to associate keys with value
        - Hash tables are popular because they typically provide low-latency lookup, insertion, and updates.
        - Applications include:
            - Mapping client IDs to counters for an API rate limiter.
            - Mapping product identifiers to product metadata in a cache.
            - Mapping document IDs to blocks or lock owners in a document store.
    - **Hash Functions**: maps a key to a value called a fingerprint.
        - In randomized analysis, a hash function is chosen randomly at the start. After that choice, it is deterministic: the same key produces the same fingerprint on repeated calls.
    - **Chained Hash tables**: A chained hash table contains *m* buckets, each storing a linked list.
        - A key is assigned to the bucket determined by its hash value modulo
        - Insertion: compute the bucket and add the key to its linked list. Inserting at the head of the list takes $O(1)$ time.

        - Query: compute the bucket and search its linked list. The cost depends on the keys already in that bucket.
            - An empty bucket can be checked quickly
            - A longer list requires more comparisons.
        - Therefore, query cost is a random variable when bucket placement depends on a randomly chosen hash function; analysis often focuses on its expected cost.
        - In the worst case, all keys can land in one bucket, making a query $O(n)$ for n stored keys. Randomized hashing is intended to make such clustering unlikely and keep lists short on average.
- **<u>Linked Lists:</u> A root with a pointer to a Node with value and location to point to next Node**
    - a fundamental linear data structure consisting of a sequence of elements where each element points to the next.
    - Unlike arrays, linked list elements are not stored in contiguous memory locations.
    - Instead, they are represented as nodes scattered across memory, dynamically connected using references or pointers
    - **Search:**
        - If root is none, return
        - Create a pointer that starts at node (root)
        - While the current pointer is not None
            - If the current pointer's value is not the target, let pointer equal pointer's next
            - Else return the pointer
    - **Insert:**
        - let pointer start at node, self.root
        - If the list is empty
            - Create a new Node with the value
            - Have root point to it next
        - If value is less than first element
            - Create a new Node with the value
            - Have self root point to new node
            - Have new node point to old first node
        - While pointer is not None
            - If value is greater than current AND less than next node value, then insert there
                - Create temp pointer to hold onto next
                - Have current point to new Node
                - New node points to temp

- **Skip Lists:** use randomized promotions to create express levels over a sorted linked list, giving expected O(log n) search time
    - **A skip list is a randomized data structure built from a sorted linked list.**
        - Analogy:
            - The bottom-level linked list is like a local subway train, stopping at every item.
            - Higher levels act like express trains, skipping items to speed up searches
        - Searching starts at a high level, moves forward while possible, and drops to lower levels when the next item would overshoot the target.
        - Insertion and Randomization
            - To insert an item:
            - Search for its correct position.
            - Insert it at the bottom level.
            - Flip a coin to decide whether to promote it to the next level.
            - Continue flipping and promoting on heads; stop at the first tails.
        - Each item's promotion decisions are independent of other items' decisions.
        - This simple randomized balancing avoids the complicated rebalancing rules of structures such as AVL or red-black trees
        - Skip lists are used in practice, including in systems that benefit from concurrency, such as key-value stores.
    - **Expected Search Cost**
        - Search cost consists of:
            - Vertical movement: one step per level
            - Horizontal movement: forward steps within each level.
        - Analyze horizontal movement by considering the search backward from its final item:
            - At a given level, the final item was promoted from the level below with probability $1/2$, yielding zero horizontal steps at that level.
            - Otherwise, the search must have taken at least one step; the same reasoning repeats for preceding items.
        - The expected horizontal steps at a level are bounded by the geometric sum
        - Thus, each level has constant expected horizontal cost.

Need to review:

1. Skip lists
2. Knapsack problem
3. Cookie distribution
4. Randomized closest pair of points
5. Log
    a. $b^c = a$ -----> $\log_b(a) = c$